FH University of Applied Sciences TECHNIKUM WIEN

# BACHELOR PAPER

Thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science in Engineering at the University of Applied Sciences Technikum Wien - Degree Program Mechatronik/Robotik

## Hybrid Coverage Path Planning for Mobile Robots using Visual Landmark-Based Workspace Reconstruction

By: Víctor Puche Alonso
Student Number: 2500330006
Supervisor 1: FH-Prof. Dr. Techn. Mohamed Aburaia, MSC

Wien, 10/06/2026

# Declaration of Authenticity

“As author and creator of this work to hand, I confirm with my signature knowledge of the relevant copyright regulations governed by higher education acts (see Urheberrechtsgesetz/ Austrian copyright law as amended as well as the Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I hereby declare that I completed the present work independently and according to the rules currently applicable at the UAS Technikum Wien and that any ideas, whether written by others or by myself, have been fully sourced and referenced. I am aware of any consequences I may face on the part of the degree program director if there should be evidence of missing autonomy and independence or evidence of any intent to fraudulently achieve a pass mark for this work (see Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I further declare that up to this date I have not published the work to hand nor have I presented it to another examination board in the same or similar form. I affirm that the version submitted matches the version in the upload tool.”

Víctor Puche i Alonso

Firmado digitalmente por Víctor Puche i Alonso
Fecha: 2026.06.10 17:59:47 +02'00'

Barcelona, 10/06/2026

Place, Date

Digital Signature

# Kurzfassung

Die vollständige Abdeckung eines Arbeitsbereichs stellt ein grundlegendes Problem der mobilen Robotik dar. Beim Coverage Path Planning (CPP) muss ein Roboter alle erreichbaren Bereiche einer Umgebung systematisch befahren und dabei Hindernisse vermeiden. Bestehende Verfahren weisen häufig Kompromisse hinsichtlich Vollständigkeit, Rechenaufwand, Online-Fähigkeit oder Sensoranforderungen auf.

In dieser Arbeit wird ein hybrides CPP-System für autonome mobile Roboter in unbekannten Umgebungen vorgestellt. Der Ansatz kombiniert eine reaktive Explorationsphase mit einer deterministischen Abdeckungsphase. Während der Exploration werden ArUco-Marker als visuelle Landmarken verwendet, um die Begrenzung des Arbeitsbereichs ohne SLAM oder kostenintensive Distanzsensoren zu rekonstruieren. Anschließend wird eine boustrophedon-inspirierte Bahnplanung erzeugt, um den Bereich systematisch abzudecken.

Die Implementierung erfolgte auf einer TurtleBot-4-Plattform unter Verwendung von ROS 2 Jazzy und MATLAB. Die Evaluierung umfasste zehn autonome Testmissionen in einer Laborumgebung. Dabei wurde eine durchschnittliche Abdeckung von 99,0 % erreicht. Die Ergebnisse zeigen, dass die Kombination aus visueller Arbeitsbereichsrekonstruktion und deterministischer geometrischer Zerlegung eine recheneffiziente und kostengünstige Alternative zu klassischen kartenbasierten CPP-Verfahren darstellt.

**Schlagwörter:** Coverage Path Planning, Mobile Robotik, ArUco-Marker, Autonome Navigation, Arbeitsbereichsrekonstruktion

# Abstract

Coverage Path Planning (CPP) is a fundamental problem in mobile robotics, requiring a robot to systematically traverse all reachable areas of a workspace while avoiding obstacles. Existing CPP approaches often involve trade-offs between coverage completeness, computational complexity, online operation, and sensing requirements.

This thesis presents a hybrid CPP system for autonomous mobile robots operating in unknown environments. The proposed method combines a reactive exploration phase with a deterministic coverage phase. During exploration, visual ArUco markers are used as spatial landmarks to reconstruct the workspace boundary without relying on SLAM or high cost ranging sensors. Once the workspace has been identified, a boustrophedon-inspired coverage path is generated to systematically cover the area.

The approach was implemented on a TurtleBot 4 platform using ROS 2 Jazzy and MATLAB and evaluated through ten autonomous coverage missions in a laboratory environment. Performance was assessed using coverage completeness, path efficiency, redundancy, energy consumption, and mission duration. The system achieved a mean coverage ratio of 99.0%, demonstrating reliable near-complete coverage while requiring only a single RGB camera and wheel odometry.

The results indicate that visual landmark-based workspace reconstruction combined with deterministic geometric decomposition provides a computationally efficient and low-cost alternative to traditional map-based CPP methods in structured environments.

**Keywords:** Coverage Path Planning, Mobile Robotics, ArUco Markers, Autonomous Navigation, Workspace Reconstruction

# Acknowledgements

I would like to express my sincere gratitude to my supervisor, FH-Prof. Dr. Techn. Mohamed Aburaia, MSc, for his guidance, feedback, and support throughout the development of this thesis.

I would also like to thank Professor Albert Massip of the Universitat Politècnica de Catalunya (UPC ESEIAAT) for his advice and assistance on several technical aspects of the project. His support and willingness to share his expertise were greatly appreciated. I am equally grateful to Lab Professor Lukas Muster for his help during the practical implementation and testing phases of this work.

Finally, I would like to thank my parents for their continuous support throughout my studies and for encouraging me to pursue my academic and professional goals. I am especially grateful to my girlfriend for her patience, understanding, and encouragement during the many hours dedicated to the completion of this thesis, especially during the tenure of our long-distance relationship. Their support was invaluable throughout this journey.

# Table of Contents


1 Introduction ..... 6
1.1 The Rise of Mobile Robotics ..... 6
1.2 Application Domains ..... 6
1.3 Coverage Path Planning (CPP) ..... 7
1.4 Challenges of Online Coverage Path Planning ..... 7
1.5 Objectives of the Thesis ..... 7
1.6 Work Planning ..... 8

2 Definition of the Coverage Path Planning Problem ..... 8
2.1 The Workspace and Configuration Space ..... 8
2.2 The Coverage Objective ..... 9
2.2.1 Defining and Measuring the Quality of Coverage ..... 10
2.2.2 Problem Statement ..... 11

3 State of the Art ..... 12
3.1 Cellular Decomposition Coverage ..... 12
3.2 Grid-Based Coverage ..... 13
3.2.1 Wavefront Algorithm ..... 14
3.2.2 Spiral-STC Algorithm ..... 15
3.3 Heuristic Coverage ..... 16
3.3.1 Randomized and Chaotic Algorithms ..... 16
3.3.2 Reactive and Potential Field Methods. ..... 17
3.3.3 Frontier-based Exploration. ..... 17
3.3.4 Wall-Following Algorithms. ..... 18
3.4 Sampling-Based Coverage ..... 18
3.5 Alternative algorithms ..... 19
3.5.1 Genetic Algorithms ..... 19
3.5.2 Particle Swarm Optimization ..... 19
3.5.3 Ant Colony Optimization ..... 20
3.6 Comparison & Conclusions ..... 20

4 Design and Implementation of the Proposed System ..... 21
4.1 System Architecture and Integration ..... 21

4.2 Exploration Phase ..... 22
4.3 Coverage Phase ..... 23
4.3.1 Boundary Reconstruction ..... 23
4.3.2 Boustrophedon Execution ..... 24
4.4 Operational Assumptions and Limitations ..... 25
4.5 Experimental Setup ..... 25
4.5.1 Camera Intrinsic Calibration ..... 25
4.5.2 Swath Width and Footprint Geometry ..... 26
4.5.3 Testing Environment ..... 26

5 Results, Discussion and Conclusions ..... 27
5.1 Experimental Results and Analysis ..... 27
5.1.1 Coverage Completeness (η) ..... 28
5.1.2 Path Efficiency (ε) ..... 28
5.1.3 Coverage Redundancy (ρ) ..... 29
5.1.4 Energy Consumption (E) ..... 29
5.1.5 Mission Time and Average Speed (T) ..... 30
5.2 Discussion ..... 30
5.2.1 Interpretation of Results ..... 30
5.2.2 Comparison with the State of the Art ..... 30
5.3 Conclusions ..... 31
5.3.1 Summary of Contributions ..... 31
5.3.2 Achievement of Research Objectives ..... 31
5.3.3 Recommendations for Future Work ..... 31

Bibliography ..... 32

List of Figures ..... 33

List of Tables ..... 34

List of Abbreviations ..... 34

Documentation table of AI-based tools ..... 35

A: Repository Access and Version Control ..... 35

# 1 Introduction

Over the last few decades, mobile robotics has expanded from strictly controlled industrial environments into dynamic, human-centric spaces like hospitals and agricultural fields. This transition has been largely driven by advances of hardware, specifically the decreasing cost of sensors, improved battery densities, and the rise of power-efficient embedded computing. However, regardless of their specific application, autonomous platforms face a shared operational challenge: the need to systematically traverse their entire environment. In robotics, this requirement is defined as Coverage Path Planning (CPP).

## 1.1 The Rise of Mobile Robotics

Since the mid-1990s, robotics has undergone a fundamental transformation. Robots once confined to rigid assembly lines are now expected to act autonomously in open, human-shared environments. According to Research Nester, the global smart robots market exceeded 23.92 billion USD and is projected to reach over 214 billion USD by 2035, driven largely by growth in the healthcare sector [1]. This expansion is enabled by the falling cost of sensors, improvements in battery energy densities originally developed for electric vehicles, and the availability of power-efficient embedded computers capable of running perception, planning, and control algorithms in real time. As a result, robots can now sense, interpret, and adapt to unknown situations without requiring a pre-programmed environment or routine.

## 1.2 Application Domains

Robotics has penetrated diverse commercial and industrial markets [3] [4], even cleaning [2]. In precision agriculture, mobile platforms monitor crop health, detect pest infestations, and optimize irrigation [3]. An autonomous drone that flies over a field and generates stress maps is fundamentally solving a coverage problem: every point in the field must be observed to build a complete report. This approach can raise productivity by up to 30 % and lower operational costs by 15 % [4].

Critical infrastructure, power transmission lines, oil and gas pipelines, wind turbines, requires regular inspection to detect corrosion or structural fatigue, an obligation now codified in law like in the German KRITIS-Dachgesetz passed on 29 January 2026 [5], [6]. These assets are often distributed over vast, hazardous areas. As Wong et al. note in their overview of robotics for harsh environments, “in many situations, the associated environment is either too dangerous or inaccessible to humans,” which motivates the deployment of robots for inspection and infrastructure management [7]. Whether a drone inspects a high wind turbine or a narrow pipeline, the path must leave no section unobserved.

## 1.3 Coverage Path Planning (CPP)

Every application above shares the need for a path that passes over all points in a workspace. In robotics literature this is the Coverage Path Planning problem. Classical motion planning computes a collision-free path from an initial to a final configuration [8]. CPP shifts the objective to covering every point of the free space. Tan et al. define CPP as the determination of a path that covers all points while detecting and avoiding obstacles, typically minimizing path length, overlap, and energy consumption [9].

Choset's foundational survey organizes CPP algorithms by coverage guarantee into heuristic (no completeness guarantee) and complete (every point visited). Complete methods are further divided by decomposition strategy: exact, approximate, or semi-approximate cellular decomposition [10]. Agmon et al. later distinguished offline CPP, where a prior map is available, from online CPP, where the path is constructed during execution without any prior knowledge of the area [11].

## 1.4 Challenges of Online Coverage Path Planning

Online CPP is difficult for several reasons. The robot does not know the geometry beforehand; it must explore and cover simultaneously. Decisions happen in real time, often ten times per second or faster. Sensors provide noisy, partial data. A moving obstacle can invalidate a path instantly. And finally, battery life and computing power are limited, so the algorithm must be efficient and lightweight. These challenges establish the evaluation parameters for any online CPP solution: path efficiency, energy consumption, computational complexity, and robustness.

## 1.5 Objectives of the Thesis

The main objective of this thesis is to design and validate a computationally efficient Coverage Path Planning (CPP) algorithm for autonomous mobile robots operating in unknown environments, using minimal sensing requirements. The specific objectives are the following:

- Develop an online CPP strategy that does not require prior environment mapping
- Minimize computational complexity to enable real-time execution
- Reduce sensor dependency using visual markers
- Evaluate the performance of the proposed method based on the metrics defined in the previous section
- Compare the proposed approach with state-of-the-art CPP methods

## 1.6 Work Planning

The work is organized into six sequential phases: literature review, system design, algorithm development, implementation in simulation, testing and evaluation, and documentation. A Gantt chart (Figure 1) details the project timeline, ensuring logical progression from theoretical analysis to validated results.

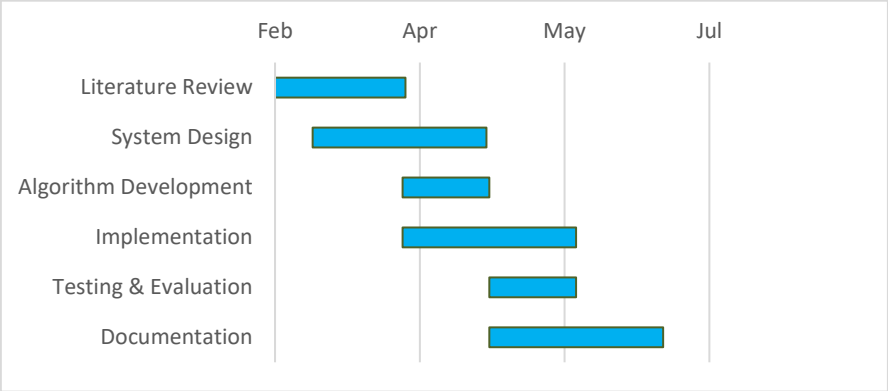



Figure 1: Gannt Diagram of the thesis schedule

# 2 Definition of the Coverage Path Planning Problem

## 2.1 The Workspace and Configuration Space

Before formally defining the coverage path planning problem, we must first establish a mathematical representation of the environment in which the robot operates. In robotics, the physical world in which the robot exists is referred to as the workspace, as defined by Lynch and Park. [12], whilst the task space is a space where the robot task can be naturally expressed. Per example, if our robot is a UAV and must cover an area through CPP at a certain height, the task space is defined as $R^3$, and if our robot is simply covering an area by rolling through whilst a camera is pointed to the ground, the task space is defined as $R^2$. For this thesis, we restrict our task space to $R^2$, since we are seeking to create an algorithm to cover an area and not a volume.

Therefore, we define the workspace as a subset of a finite Euclidean plane, $W \subset R^2$, here, $W$ represents the total area of interest, for our purposes, a region that requires coverage. This area can be a floor, the boundaries of a crop field, or the search area in a disaster zone. Obstacles $O \subset W$ are closed sets with non-empty interior that the robot cannot traverse. The region where the robot can move is the free space, which we define as the complement between the obstacles set within the workspace:

$$F = W \setminus O$$

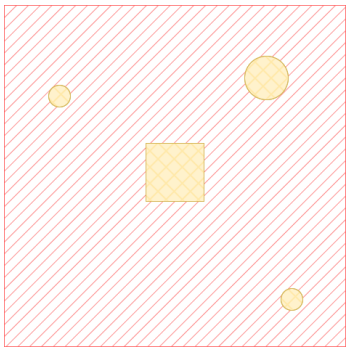


Figure 2: Representation of a R^2 workspace (red) with obstacles (yellow).

However, the robot itself occupies physical space. To account for its dimensions, we introduce the configuration space *C* (C-space), an idea introduced by Lozano-Pérez [13]. A configuration $q \in C$ represents a possible pose (position and orientation) of the robot. The robot's path over time is a continuous sequence $q(t) \rightarrow t \in [0, T]$ where $q(t)$ is the configuration at a given time. For a circular robot of radius *r*, a configuration is collision-free if the robot's body does not intersect any obstacle. This is equivalent to treating the robot as a point and expanding each obstacle by *r*. The set of all collision-free configurations is denoted as $C_{free} \subset C$

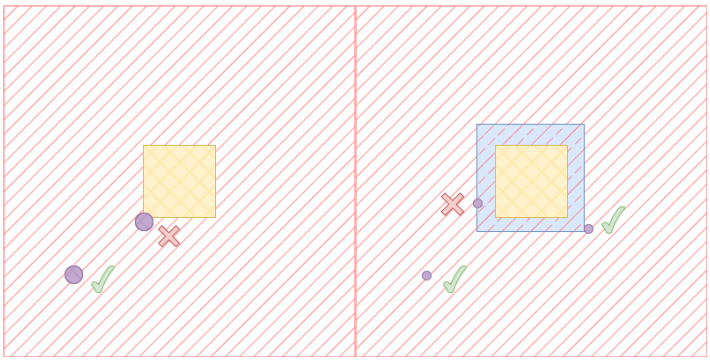


Figure 3: Representation of a R^2 workspace with obstacles and robot (purple).

The results obtained through configuration space analysis match those obtained through direct geometric collision checking, but the configuration space method reduces the problem to a simple point-in-polygon test, which reduces computational load by a lot.

## 2.2 The Coverage Objective

Coverage means passing a sensor over every point in free space. Let the robot's configuration at a time *t* be indicated by $q(t) \in C$, and $S(q) \subset R^2$ the camera footprint on the ground. A point $p \in F$ is covered if $\exists t \in [0, T]: p \in S\big(q(t)\big)$. The covered set is $C_{covered} = \cup_{t \in [0,T]} S\big(q(t)\big)$.

Complete coverage is achieved when $F \subseteq C_{covered}$. In other words, every free point has been observed at least once.

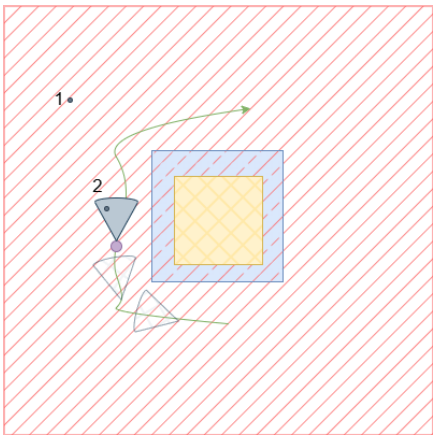



Figure 4: Representation of a R^2 workspace with obstacles, a robot, an expanded obstacle area, representations of footprints, and the trajectory q(t).

### 2.2.1 Defining and Measuring the Quality of Coverage

Defining complete coverage as $F \subseteq C_{covered}$ establishes a binary condition: either coverage is complete, or it is not. However, to compare algorithms in practice, we use the following metrics:

Table 1: Performance metrics for coverage path planning evaluation

| Metric | Symbol | Formula | Description |
|---|---|---|---|
| Coverage ratio | $\eta$ | $\eta = \frac{\lvert C_{covered} \cap F \rvert}{\lvert F \rvert} \cdot 100\%$ | Percentage of free space covered. 100% is ideal. |
| Redundancy | $\rho$ | $\rho = \frac{(\sum \lvert S(q) \rvert - \lvert C_{covered} \rvert)}{\lvert C_{covered} \rvert}$ | Overlap relative to covered area. $p = 0$ means no overlap, |
| Path efficiency | $\epsilon$ | $\epsilon = \frac{\lvert \pi' \rvert}{\lvert \pi \rvert}$ | Ratio between the ideal path length (π′) and the actual path length (π). Values closer to 1 indicate more efficient trajectories. |
| Energy efficiency | $\xi$ | $\xi = \frac{E_{min}}{E}$ | Ratio between the theoretical minimum energy required to |

| | | | |
|---|---|---|---|
| | | | follow π′ and the actual energy consumed while executing π. |
| Robustness | – | Success rate, computational time | Empirical measures: Percentage of successful trials, time per path calculation. |

Where $\pi$ is the real route and $\pi'$ is the most optimal route and $E_{min}$ is the energy required to follow $\pi'$, $\text{assuming } E = \alpha \cdot |\pi|$, where α is an energy-per-distance constant, specific to the robot that must be estimated on an individual basis

## 2.2.2 Problem Statement

With all our metrics defined, we can state the issue to solve with CPP mathematically:
Given an unknown environment $W \subset R^2$ containing obstacles $O \subset W$, and a mobile robot with a camera footprint $S(q)$ and a configuration space $C$, the online coverage path planning problem is to find a control policy that generates a path $q(t)$ for $t \in [0, T]$ such that it complies with the following conditions:

1. Coverage Completeness: $F \subseteq \cup_{t\in[0,T]} S\big(q(t)\big)$ where $F = W \setminus O$
2. Collision Avoidance: $q(t) \in C_{free}$ for all $t$
3. Online operation: The path is generated only through real-time sensor data, no prior mapping.
4. Efficiency: Path minimizes redundancy $\rho$, path length $\pi$, and energy consumption, $E$

Existing CPP methods involve unavoidable trade-offs between completeness, computational cost, and sensing requirements, as summarized in Table 2.

Table 2: Trade-offs of existing CPP approaches for unknown environments

| Method | Completeness | Online Abled | Computational Cost | Sensor Needs |
|---|---|---|---|---|
| **Exact Cellular** | High | No | Medium | Low (map required) |
| **Grid-Based (approximate)** | Medium | Yes | High | High (LiDAR, camera) |
| **Heuristic** | Low | Yes | Low | Very Low |
| **Learning-** | Medium | Limited | Very high | High |

| | | | | |
|---|---|---|---|---|
| **based (RL)** | | | | |

No single method simultaneously provides completeness, online operation, low computational cost, and minimal sensing. This gap motivates the hybrid approach proposed in Chapter 4.

# 3 State of the Art

This chapter presents a review of the main approaches to coverage path planning. To structure the discussion, existing methods are grouped according to how they represent the environment and generate coverage trajectories. The chapter is organized into five categories: cellular decomposition, grid-based methods, heuristic strategies, sampling-based approaches, and optimization-based techniques. Each category is examined in terms of its operating principles, advantages, and limitations.

## 3.1 Cellular Decomposition Coverage

As explained by Howie Choset [14], cellular decomposition transforms the coverage problem by subdividing the free space into smaller, geometrically simple regions called cells. The robot covers each cell with simple back-and-forth (boustrophedon) sweeps. The visitation order is determined by an adjacency graph, where nodes are cells and edges connect adjacent cells. It was first applied to motion planning by Latombe [22] using trapezoidal decomposition.

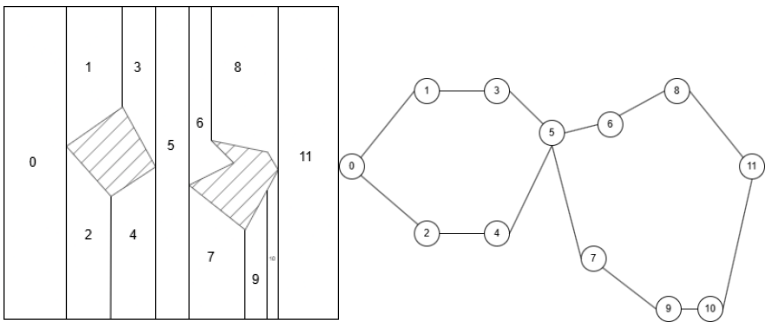



Figure 5: (a) Trapezoidal cellular decomposition of an environment (b) Trapezoidal adjacency graph

Choset's boustrophedon decomposition improves upon the classical trapezoidal decomposition by merging cells that would otherwise be unnecessarily split, resulting in fewer cells and more efficient coverage.

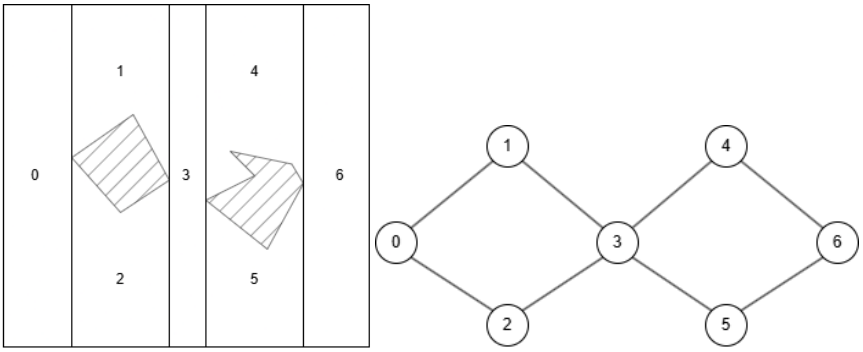



Figure 6: (a) Boustrophedon cellular decomposition of an environment (b) Boustrophedon adjacency graph

Graph traversal strategies such as Depth-First Search, Breadth-First Search and Dijkstra (DFS, BFS) are used to order cell visits. Dijkstra is preferred when cells vary in size, as it minimizes physical distance.

The main limitation of Cellular Decomposition Coverage is the requirement of a complete map beforehand. Dynamic or unknown obstacles invalidate the graph. Decomposition also can be computationally expensive for cluttered environments. These limitations motivate online grid-based methods.

## 3.2 Grid-Based Coverage

Grid-based methods represent the environment as a uniform grid of cells. Moravec and Elfes [15] introduced occupancy grids: a 2D array of $M * N$ cells, each of size $\Delta * \Delta$. Each cell stores a value in [-1,1]: negative = free, zero = unknown, positive = occupied. As the robot moves, sensor readings update these probabilities through re-examinations of the environment.

These methods are approximate cellular decompositions, explained in section 1.3. They are also resolution-complete: full coverage depends on cell size $\Delta$. While triangular cells have been proposed [16], square cells of robot size are sufficient for most platforms, as their resolution is enough.

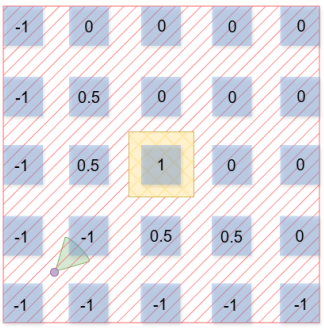



Figure 7: Representation of a grid-based cellular map

### 3.2.1 Wavefront Algorithm

The wavefront algorithm [17] is an offline grid-based method. The goal cell is assigned 0. Neighboring cells are numbered 1, then their neighbors 2, and so on, propagating outward like a wave until the start cell is reached. Obstacle cells are left unnumbered.

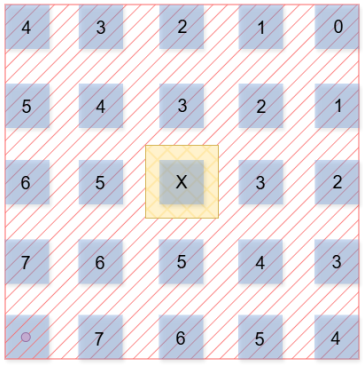



Figure 8: Representation of a wavefront map. A workspace (red) is populated by cells (blue) marked with numeralia. Cells with obstacles (yellow) are ignored. The robot (purple) sits at the bottom left.

The robot starts at the start cell and moves to the unvisited neighbor with the smallest value (descending gradient), marking each cell as visited, until it reaches the goal (0). This guarantees that each cell is visited at most once, achieving near-optimal coverage. For coverage path planning, the wavefront must be adapted to cover every single cell. One approach is the following: After reaching a goal, a new goal is selected among uncovered cells, and the wavefront is recomputed. The robot moves from its current position to the new goal, gradually covering the entire workspace

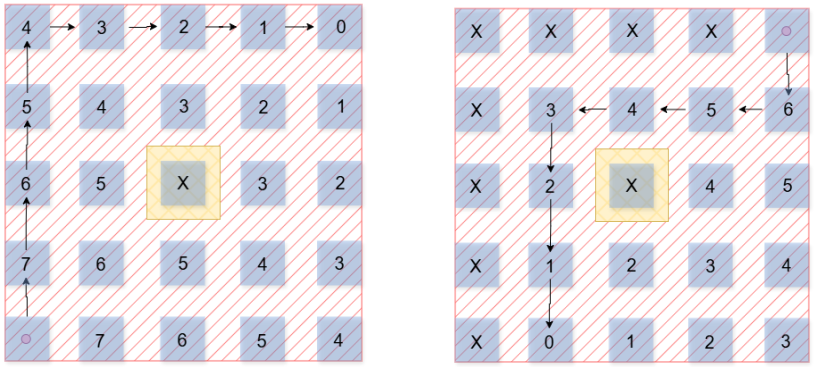



Figure 9: (a) Pathing on first iteration of the algorithm, choosing the shortest route. (b) Pathing on second iteration. Previously visited cells are marked as obstacles, and furthest cell from origin is marked as goal.

### 3.2.2 Spiral-STC Algorithm

The Spiral-STC algorithm, proposed by Gabriely and Rimon [18], is an online, sensor-based coverage path planning (CPP) method that operates on a grid representation of the workspace. The environment is divided into square cells of size $D * D$, corresponding to the robot's coverage footprint. Additionally, a higher-level grid is defined using "mega-cells" of size $2D * 2D$, forming the STC grid.

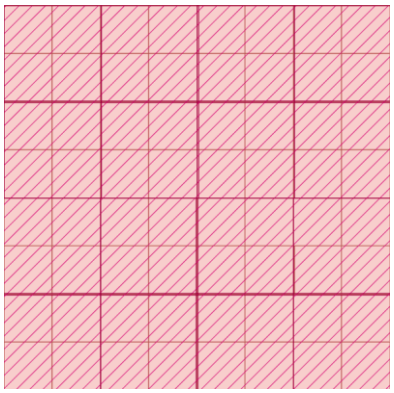


Figure 10: A workspace (red) subdivided in mega cell (pink hatch) that are subdivided by small cells (red)

Starting from its initial mega-cell, the robot explores neighboring mega-cells in a counterclockwise order. A mega-cell is considered free only if all its constituent cells are obstacle-free. When a free and unvisited neighboring mega-cell is found, it is added to a spanning tree, and the robot moves into it. If no such neighbor exists, the robot backtracks along the spanning tree. This process continues until all reachable mega-cells are visited.

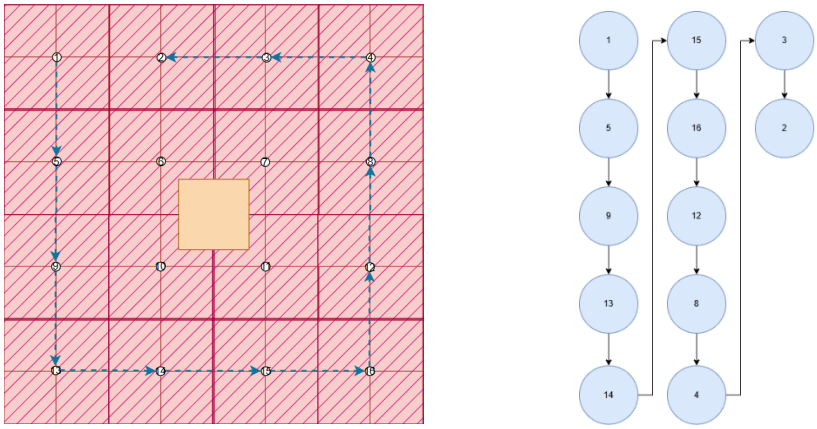



Figure 11: (a) Trajectory of a Spiral-STC algorithm that avoids megacells with obstacles inside. (b) Spanning tree node of megacells covered in the execution of this iteration

As the spanning tree is constructed, the robot follows its perimeter, ensuring that each $D * D$ cell is covered exactly once, producing a spiral-like trajectory.

Despite its flexibility, grid-based CPP methods present several limitations. Their computational and memory requirements increase with grid resolution, and the discretization of the environment may introduce inaccuracies, especially in irregular or narrow spaces. Additionally, obstacle avoidance and re-planning can lead to redundant movements, reducing efficiency compared to exact methods.

To address these limitations, alternative approaches based on heuristic rules and local sensory information have been proposed. These methods avoid explicit environment representation and aim to achieve efficient coverage with lower computational cost.

## 3.3 Heuristic Coverage

Heuristic coverage methods rely on simple reactive strategies instead of explicit environment representations or global planning. The robot follows predefined motion rules based solely on local sensor information, progressively exploring the environment with minimal computational requirements.

As surveyed by Tan et al [19], these methods encompass a wide range of approaches, which can be broadly classified.

### 3.3.1 Randomized and Chaotic Algorithms

Randomized methods use stochastic processes to guide exploration, requiring little environmental knowledge and low computational cost. A common example is the Random Walk (RW), where the robot moves in straight lines and changes direction randomly upon encountering obstacles [20]. While simple and suitable for online coverage, this approach is inefficient due to high overlap and poor path optimization.

Variants such as Brownian Motion (BM) and Lévy Flight (LF) introduce probabilistic step lengths. BM generates short, normally distributed steps, leading to localized exploration and high redundancy. In contrast, LF combines frequent short movements with occasional long jumps, improving exploration by reducing repeated coverage of the same areas.

Chaotic approaches provide deterministic yet seemingly random trajectories. A notable example is the Chaotic Coverage Path Planner, based on the Lorenz system [21].

$$\frac{dx}{dt} = \sigma(y - x)$$
$$\frac{dy}{dt} = x(\rho - z) - y$$
$$\frac{dz}{dt} = xy - \beta z$$

For standard chaotic behavior, the parameters are typically set to σ=10, ρ=28 & β=8/3. The robot's motion is derived from the system output, producing a trajectory that efficiently explores space without explicit planning. However, like other heuristic methods, it lacks optimization and may result in redundant coverage.

### 3.3.2 Reactive and Potential Field Methods.

These methods operate directly on sensor inputs without constructing explicit maps, making them suitable for real-time obstacle avoidance. A representative approach is the Artificial Potential Field (APF), which models the workspace as a virtual force field. The goal generates an attractive force, while obstacles produce repulsive forces. The robot moves according to the negative gradient of the resulting potential function, enabling smooth and reactive navigation without global path planning.

$$F = -\nabla U$$

This formulation integrates obstacle avoidance into the control loop and supports dynamic environments where the goal position may change. However, APF is affected by local minima, where attractive and repulsive forces cancel out, potentially trapping the robot in non-goal states. Although APF is effective for real-time navigation in unknown environments, it is not inherently suited for complete coverage, as it provides no guarantee of visiting all regions of the workspace. To address this, it is often combined with exploration or mapping strategies. One approach is to integrate APF with the Random Walk (RW) method (Section 3.3.1), producing a hybrid system that enables both exploration and obstacle avoidance, as demonstrated by Sutantyo et al. in a multi-robot framework [22].

### 3.3.3 Frontier-based Exploration.

Frontier-based exploration [23] is a heuristic strategy that enables a mobile robot to map an unknown environment by iteratively identifying and navigating toward frontiers, which are the boundaries between known open space and unexplored territory. This process relies on a two-dimensional occupancy grid where each cell is classified as open, occupied, or unknown. A frontier is specifically defined as any open cell adjacent to an unknown cell. The algorithm identifies these frontier cells, clusters them into connected regions, and calculates a centroid for each region to serve as a potential goal point.

The robot systematically selects the nearest accessible centroid that has not been previously visited. A path planner then computes the shortest collision-free trajectory while a reactive obstacle avoidance module ensures the robot can navigate around unmapped or dynamic hazards. Upon reaching the target centroid, the robot executes a 360° sensor sweep to gather new range data, which is then integrated into the occupancy grid to convert unknown cells into open or occupied states. This cycle of detection, selection, and navigation continues until no

frontiers remain, indicating that all reachable space has been observed and the map is complete.

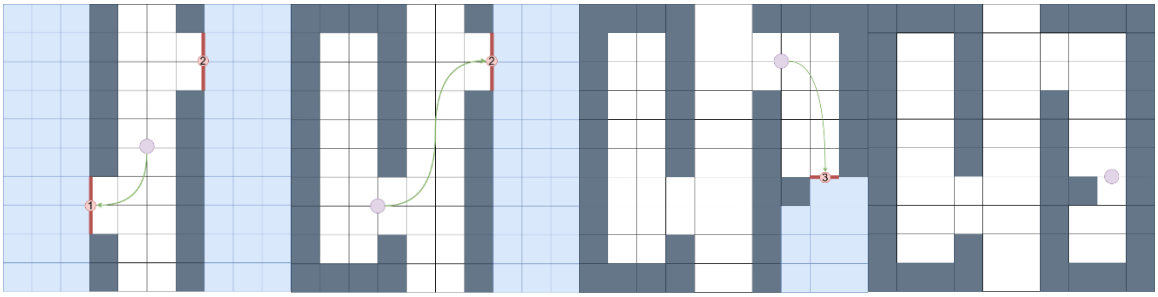


Figure 12: (a) to (d) showcase a step-by-step frontier-based exploration.

### 3.3.4 Wall-Following Algorithms.

Wall-following algorithms are reactive navigation strategies in which a robot moves along the boundaries of obstacles or the environment. In the context of Coverage Path Planning (CPP), their role extends beyond perimeter tracing, contributing to systematic area coverage.
As described by Lee et al. [24], wall-following for CPP is based on three principles. First, the robot follows physical boundaries using local sensors, ensuring coverage of edge regions. Second, previously traversed paths are treated as “virtual walls,” allowing the robot to incrementally fill adjacent unexplored areas. This results in a coverage pattern like sequential strip-based motion.

Such approaches are commonly used in commercial cleaning robots (like the iRobot Roomba), where wall-following ensures coverage near walls and around obstacles. However, these methods rely heavily on local decisions and may lead to inefficient coverage patterns in complex environments.

## 3.4 Sampling-Based Coverage

Sampling-based coverage path planning methods construct a representation of the free space through random sampling, avoiding explicit geometric decomposition or fixed grids. Instead, they incrementally build a graph or tree of feasible configurations to guide coverage.

These approaches originate from motion planning, particularly algorithms such as Probabilistic Roadmaps (PRM) and Rapidly exploring Random Trees (RRT). When adapted to coverage, the objective shifts from reaching a single goal to ensuring that the robot’s sensor footprint spans the entire workspace.

In PRM-based coverage, sampled configurations are connected into a collision-free roadmap. Coverage is achieved by ensuring that the combined footprints of selected nodes span the

environment, reducing the problem to a graph traversal task, often formulated as a Traveling Salesman Problem (TSP) variant.

A key limitation of uniform sampling is poor coverage of narrow or small regions. This can be mitigated through adaptive sampling, where uncovered areas are iteratively identified and resampled.

RRT-based approaches grow a tree incrementally from the robot's initial position, naturally supporting online operation. Each new node contributes to coverage through its footprint. However, since RRT is designed for exploration, it often produces redundant paths and does not guarantee efficient coverage. Variants such as RRT* improve path quality at the cost of increased computational complexity.

Overall, sampling-based methods offer flexibility in complex or high-dimensional environments but ensuring complete and efficient coverage typically requires additional mechanisms to track explored regions and reduce redundancy.

## 3.5 Alternative algorithms

### 3.5.1 Genetic Algorithms

Genetic algorithms frame path planning as an evolutionary optimization problem. Candidate trajectories, encoded as sequences of waypoints or control points, are evaluated against a fitness function that prioritizes spatial coverage while penalizing overlapping and obstacle collisions. Through iterative selection, crossover, and mutation of these path segments, the algorithm gradually converges on an optimal route. While highly adaptable to complex constraints without requiring gradient information, this approach typically demands significant computational overhead and risks premature convergence to suboptimal paths.

### 3.5.2 Particle Swarm Optimization

Particle Swarm Optimization adapts to coverage path planning by treating each candidate path as a particle moving through a high-dimensional solution space, where the particle's position encodes a sequence of waypoints, control points, or grid-cell transitions. Each particle maintains its own best-known path (personal best) and the swarm collectively tracks the best path found so far (global best). At each iteration, particles update their trajectories by combining a tendency to return to their personal best with a tendency to move toward the global best, weighted by random coefficients that encourage exploration. The fitness of a path is evaluated by simulating coverage completeness, path length, overlap, and obstacle compliance, guiding the swarm toward efficient, thorough trajectories.

This approach is particularly attractive because it requires no gradient information, handles complex objective functions naturally, and often converges faster than genetic algorithms on continuous or smoothly varying path representations.

However, PSO is susceptible to premature convergence if the swarm loses diversity, and it can struggle with discrete or highly constrained search spaces unless the encoding and update rules are carefully designed.

### 3.5.3 Ant Colony Optimization

Ant Colony Optimization applied to coverage path planning treats the environment as a graph, typically a grid of cells or a decomposition into regions, where edges represent feasible movements or coverage actions. Artificial ants construct paths incrementally by moving from node to node, selecting the next edge with a probability that balances pheromone concentration (indicating historical success) with a heuristic bias toward uncovered areas or shorter steps. Once a path is completed, its fitness is evaluated based on coverage completeness, path length, and overlap, and pheromone levels are updated accordingly: edges used in high-quality paths receive additional pheromone, while evaporation reduces the influence of inferior solutions over time. Through repeated iterations, the colony converges on a set of trails that collectively cover the workspace efficiently, often yielding near-optimal routes that implicitly handle obstacles and irregular boundaries. This approach is particularly well-suited for coverage problems that can be framed as graph traversal, and it benefits from positive feedback and parallel exploration.

However, ACO is sensitive to parameter choices, can stagnate prematurely if pheromone diversity is lost, and may require careful graph construction to ensure complete coverage, limitations that can be overcome by hybridizing the system with other algorithmic designs.

## 3.6 Comparison & Conclusions

The reviewed approaches show that coverage path planning in unknown environments involves unavoidable trade-offs between completeness, computational cost, and sensing requirements. No existing method satisfies all these constraints simultaneously.

Table 3: Comparisons of the different categories of CPP methods

| Method | Completeness | Online Abled | Computational Cost | Sensor Needs |
|---|---|---|---|---|
| **Cellular** | High | No | Medium | Low |
| **Grid-Based** | Medium | Yes | High | High |
| **Heuristic** | Low | Yes | Low | Low |

| | | | | |
|---|---|---|---|---|
| **Sampling-Based** | Medium | Limited | High | High |
| **Optimization-Based** | Medium | No | Very High | High |

As shown in Table 3, no existing approach simultaneously satisfies complete online operation, low computational cost, and minimal sensing requirements.

No existing approach simultaneously satisfies all four requirements derived from the challenges of online operation discussed in chapter 2: completeness, online capability, low computational cost, and minimal sensor reliance. In applications such as precision agriculture, infrastructure inspection, and search and rescue, there is a clear need for a CPP algorithm that can operate without a prior map, provides a guarantee of complete coverage, runs on resource-constrained hardware and relies on low-cost, lightweight sensorics. This limitation motivates the approach proposed in the next chapter, where we attempt to bridge different implementations discussed in this chapter by disseminating it into microsystems, reducing algorithms into a set of features that we can pick and choose from to create our own personalized algorithm.

# 4 Design and Implementation of the Proposed System

This chapter details a hybrid Coverage Path Planning (CPP) system that transforms an unknown environment into a partially structured geometric workspace using visual landmarks. The system prioritizes real-time feasibility on resource-constrained hardware by trading off universal autonomy for deterministic geometric decomposition.

## 4.1 System Architecture and Integration

The implementation utilizes a TurtleBot 4 mobile platform integrated via ROS 2 Jazzy. To leverage high-level matrix operations and rapid prototyping, the control logic resides in MATLAB, communicating with the robot over a Linux-based middleware. We utilize a MATLAB-ROS 2 bridge rather than native C++ nodes to prioritize algorithmic transparency and ease of geometric visualization, accepting a marginal increase in communication latency which is non-critical for low-speed coverage tasks.

The system begins with an Online Exploration Phase to define the workspace boundaries. The robot employs a wall-following heuristic to maximize the probability of encountering environmental boundaries within a time limit $T_{max}$. To account for non-modeled obstacles or sensor occlusion, a tactile safety layer was implemented using the TurtleBot 4 bumper. Upon collision,

the system interrupts the MATLAB global planner and executes a deterministic escape sequence: a 1.5s reverse maneuver at -0.25 m/s, followed by a 180° rotation to clear the path before resuming the coverage trajectory. This safety layer overrides both modes of behavior, exploration and coverage, and marks the bump on the world map in the GUI of the algorithm.

The general behavior of the algorithm is explained through the following pseudocode:

```
INPUT:
    RGB image stream C
    Depth stream D
    Odometry data O
    Target marker count N
OUTPUT:
    Autonomous coverage execution

Initialize ROS2 communication
Initialize robot state
Initialize marker database

WHILE mission active DO
    Read odometry data
    Read RGB image
    Read depth image
    IF bumper collision detected THEN
        Execute emergency recovery
        CONTINUE
    END IF
    IF current state is EXPLORATION THEN
        Execute exploration behavior
        Detect ArUco markers
        FOR each detected marker DO
            Estimate marker position
            Transform coordinates into world frame
            Store marker location
        END FOR
        IF detected marker count ≥ N THEN
            Generate coverage path
            Switch state to COVERAGE
        END IF
    ELSE IF current state is COVERAGE THEN
        Follow generated waypoints
        Apply reactive obstacle avoidance
    END IF
END WHILE
```

## 4.2 Exploration Phase

Instead of computationally expensive SLAM (Simultaneous Localization and Mapping), the system uses ArUco markers as discrete spatial anchors. This eliminates the "Data Association" problem; each marker provides a unique ID, preventing map corruption from sensor noise or

aliasing. Using the pinhole camera model, detections in the camera frame $x_c$ are projected into a global world frame $X_w$ initialized at the robot's starting pose:

$$X_w = [R|t]_{robot} \cdot [R|t]^{-1}_{camera} \cdot x_c$$

We also introduce a constraint: A minimum of $N \geq 3$ markers are required to transition to the Coverage Phase. If $N < 3$ at $T_{max}$, the system terminates, as a valid polygon cannot be formed.

```
INPUT:
    RGB image stream C
    Robot pose Pr
OUTPUT:
    Registered marker set M

Initialize empty marker database M
WHILE exploration active DO
    Move robot forward
    IF exploration timeout reached THEN
        Rotate robot in place
    END IF
    Capture RGB image
    Detect ArUco markers
    FOR each detected marker mi DO
        IF marker is not already registered THEN
            Estimate marker distance
            Compute marker position in camera frame
            Transform marker coordinates into robot frame
            Transform robot-frame coordinates into world frame
            Store marker in M
        END IF
    END FOR
END WHILE
RETURN M
```

## 4.3 Coverage Phase

Once markers are registered, the system executes offline decomposition. This transition from "reactive" exploration to "deterministic" coverage ensures completeness within the defined polygon.

### 4.3.1 Boundary Reconstruction

The unordered set of markers $P$ is sorted by polar angle $\theta$ relative to the geometric centroid *c*:

$$c = \frac{1}{N}\sum p_i,\ \theta_i = atan2(y_i - c_y, x_i - c_x)$$

Assuming a convex polygon allows for a unique, non-self-intersecting boundary reconstruction using simple polar sorting. Afterwards, the workspace area is calculated via the Shoelace Formula.

$$A = \frac{1}{2}\left|\sum_{k=1}^{N}(x_k y_{k+1} - x_{k+1}y_k)\right|$$

The polygon is then partitioned into *K* parallel swaths of fixed width $w_s$, where $w_s$ is the camera's effective sensor footprint. A scanline intersection algorithm identifies $x_{min}$ and $x_{max}$ for each vertical increment, $y_k$.

$$R_k = [x_{min}^{(k)}, x_{max}^{(k)}] \times [y_k, y_k + w]$$

The computational cost scales linearly with the number of scanlines and polygon edges. This is significantly more memory-efficient than occupancy-grid methods which scale with environment resolution.

## 4.3.2 Boustrophedon Execution

The swaths are converted into a sequence of waypoints. To minimize unnecessary turning manoeuvres (energy cost), the path follows an alternating sequence.

- Odd Swaths: $x_{min} \rightarrow x_{max}$
- Even Swaths: $x_{max} \rightarrow x_{min}$

```
INPUT:
    Marker positions P
    Swath width w
    Safety margin s
OUTPUT:
    Coverage trajectory T

Compute workspace boundaries from P
Expand workspace using safety margin s
Compute number of coverage passes
Generate parallel swaths separated by w
FOR each generated swath i DO
    IF i is odd THEN
        Generate left-to-right traversal
    ELSE
        Generate right-to-left traversal
    END IF
    Append generated segment to T
END FOR
FOR each waypoint in T DO
    Compute heading error
```

```
    Move robot toward waypoint
    IF waypoint reached THEN
        Advance to next waypoint
    END IF
END FOR
RETURN T
```

## 4.4 Operational Assumptions and Limitations

To maintain depth, the following engineering constraints are acknowledged:

1. **Convexity:** The algorithm assumes markers define a convex shape. Non-convex layouts will result in path segments outside the intended workspace
2. **Odometry Stability:** The global coordinate consistency relies on the TurtleBot's wheel odometry; drift over long durations may lead to swath misalignment.
3. **Static Environment:** The path is computed offline; dynamic obstacles during the coverage phase require reactive pausing rather than path re-planning.

## 4.5 Experimental Setup

### 4.5.1 Camera Intrinsic Calibration

Precise coverage depends on the accuracy of the coordinate transformation from the 2D image plane to the 3D world frame. We employ the Zhang (2000) method to estimate the intrinsic matrix *K*. This procedure is critical for correcting lens distortion that would otherwise warp the reconstructed polygon boundary. Using a 25mm chessboard target and the MATLAB Camera Calibrator, we solve for:

$$K = \begin{bmatrix} fx & 0 & c_x \\ 0 & fy & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

We target a mean reprojection error of < 0.5 pixels. This provides sufficiently accurate marker localization, preventing gaps in the coverage strips. Results of the calibration are shown in the following figures.

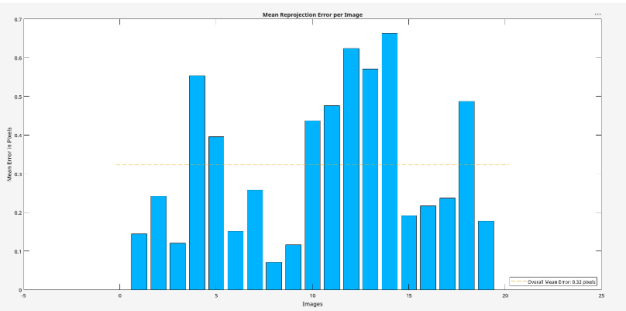



Figure 13: Mean Reprojection Error per Image, showing an overall mean error of 0.32 pixels., within the parameters of acceptance

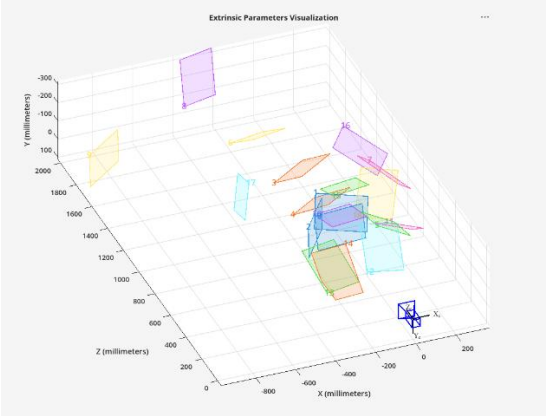



Figure 14: Visualization of the scanned checkerboards taken on calibration in a 3D plot

### 4.5.2 Swath Width and Footprint Geometry

The swath width, $w_s$, is the primary constraint for the Riemann-style decomposition. Because the OAK-D is forward-mounted with a downward tilt, the ground projection is trapezoidal. $w_s$ is defined as the lateral width at the farthest distance where a 20cm ArUco marker can be reliably scanned.

For an operational height of $H$ = 0.25 m and a tilt of α = 14.9º, Given the horizontal FOV of 69°, the theoretical maximum lateral coverage is 2.06 m. However, to ensure high-confidence ArUco detection, the system utilizes a dynamic swath width capped at 1.2 meters.

### 4.5.3 Testing Environment

To validate the proposed system, we established a controlled testing environment designed to simulate a constrained indoor workspace. Tests were conducted in a laboratory setting with a flat, level floor to minimize odometrical noise, in laboratory C1.08 of the Fachhochschule Technikum Wien. Six ArUco markers using dictionary 6x6 and a size of 16.5 cm were placed across the floor of the laboratory, simulating a convex workspace area. Uniform overhead lighting via fluorescent lightning was maintained to ensure a high Signal-to-Noise Ratio (SNR) for the ArUco detection pipeline, minimizing jitter in the pose estimation.

The following constants were utilized across all experimental runs to ensure repeatability. The reasoning for the velocities is justified on a need to reduce motion blur on the RGB stream, which is crucial to detect proper markers.

1. Linear Velocity $v$: 0.2 m/s
2. Angular Velocity $\omega$: 0.5 rad/s

The environment around the markers was heavily constrained to avoid the robot getting stuck in areas irrelevant to the testing ground due to its random-movement philosophy.

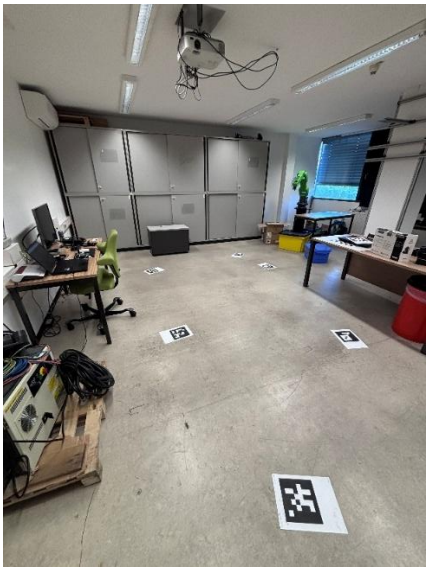


Figure 15: Picture of the testing environment for the algorithm

# 5 Results, Discussion and Conclusions

## 5.1 Experimental Results and Analysis

A total of ten autonomous coverage missions were conducted using the TurtleBot 4 platform. The following table summarizes the key performance metrics for all ten missions.

Table 4: Performance metrics for the ten missions

| Mission | $\eta$ (%) | $\epsilon$ | $\rho$ | $\pi'(m)$ | $\pi(m)$ | $E(J)$ | $T(s)$ |
|---|---|---|---|---|---|---|---|
| 1 | 99.6 | 0.262 | 0.399 | 9.24 | 35.25 | 2644 | 451.8 |
| 2 | 100.0 | 0.079 | 0.953 | 8.54 | 107.51 | 8063 | 1223.9 |
| 3 | 99.8 | 0.307 | 1.644 | 5.81 | 18.90 | 1418 | 377.6 |
| 4 | 96.2 | 0.247 | 0.541 | 8.67 | 35.04 | 2628 | 371.6 |
| 5 | 98.4 | 0.270 | 0.989 | 9.26 | 34.31 | 2573 | 429.1 |
| 6 | 97.5 | 0.272 | 0.825 | 9.49 | 34.91 | 2618 | 466.3 |
| 7 | 100.0 | 0.136 | 0.808 | 8.84 | 65.18 | 4889 | 846.5 |
| 8 | 99.2 | 0.096 | 0.752 | 9.45 | 98.32 | 7374 | 1144.2 |
| 9 | 100.0 | 0.326 | 0.736 | 9.37 | 28.77 | 2158 | 277.5 |
| 10 | 99.9 | 0.183 | 0.622 | 9.77 | 53.37 | 4003 | 533.9 |
| Mean | 99.0 | 0.218 | 0.827 | 8.84 | 51.16 | 3837 | 612.2 |
| Std Dev | 1.21 | 0.090 | 0.342 | 0.96 | 30.7 | 2302 | 324.7 |

### 5.1.1 Coverage Completeness (η)

Across all ten missions, the system achieved a mean coverage ratio of 99.0% (σ = 1.2%), with a minimum of 96.2% and a maximum of 100%. Figure 15 illustrates the coverage ratio per mission. This demonstrates that the proposed hybrid approach reliably covers nearly the entire defined workspace, even when path efficiency is compromised.

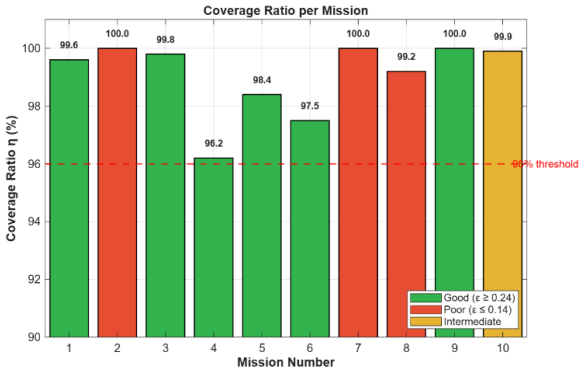



Figure 16:Coverage ratio (η) per mission

### 5.1.2 Path Efficiency (ε)

The most significant finding concerns path efficiency ε. The results reveal a bimodal distribution, where six missions (60%) exhibited good efficiency, with ε ranging from 0.247 to 0.326. In these cases, the robot travelled approximately 3.7 times the optimal path length. Three missions (30%) showed poor efficiency, with ε ranging from 0.079 to 0.136. Here, the robot travelled 9–12 times the optimal path length. One mission (10%) fell into an intermediate range, ε = 0.183.

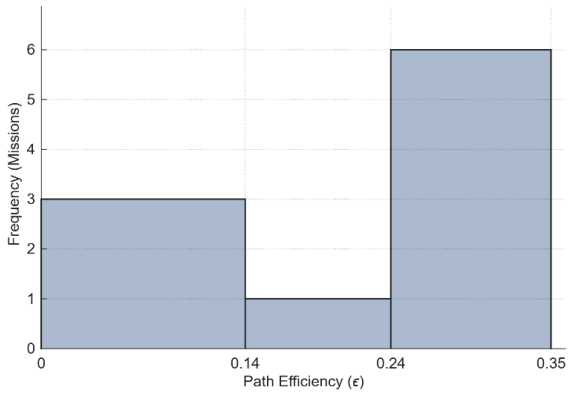



Figure 17: Path efficiency ε frequency histogram

### 5.1.3 Coverage Redundancy (ρ)

Coverage redundancy (ρ) ranged from 0.399 to 1.644, with a mean of 0.827 (σ = 0.342). This means that, on average, the robot scanned 83% more area than strictly necessary. Notably, redundancy did not correlate with path efficiency ($R^2 = 0.03$) as shown in figure 18. High redundancy occurred primarily in missions with smaller optimal path lengths, like in the case of mission 3 ($\pi' = 5.81, \rho = 1.644$), where the fixed swath width caused proportionally greater overlap. This indicates that redundancy is driven primarily by polygon geometry rather than navigation quality.

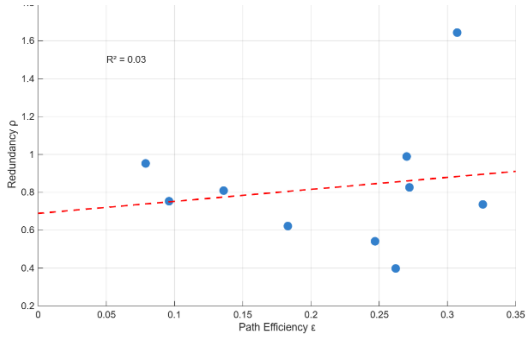



Figure 18: Redundancy (ρ) versus path efficiency (ε). No correlation exists

### 5.1.4 Energy Consumption (E)

Based on the empirical energy model, consumption varied from 1418 J (Mission 3, $\varepsilon$ = 0.307) to 8063 J (Mission 2, $\varepsilon$ = 0.079), with a mean of 3837 J (σ = 2302). As illustrated in Figure 19, energy expenditure scales inversely with path efficiency: low-efficiency missions ($\varepsilon \leq 0.14$) consumed 3 to 5 times more energy than high-efficiency runs ($\varepsilon \geq 0.24$). At the extremes, the least efficient run (Mission 2) consumed 12.6 times its theoretical minimum (8063 J actual vs. 641 J ideal), while the most efficient run (Mission 9, $\varepsilon$ = 0.326) required only 3.1 times its ideal baseline (2158 J vs. 703 J). This strict linear scaling confirms that TurtleBot 4 energy consumption appears to be dominated primarily by total travel distance, with negligible overhead introduced by turning or stopping manoeuvres, with negligible overhead introduced by turning or stopping manoeuvres.

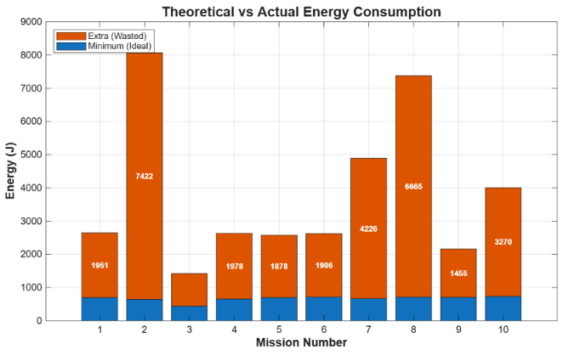



Figure 19: Theoretical vs Actual Energy Consumption of the 10 missions

### 5.1.5 Mission Time and Average Speed (T)

Mission time scaled linearly with actual distance ($R^2 = 0.95$), confirming a constant average speed of approximately 0.09 m/s (Figure 18). This speed is limited by the need to reduce motion blur for ArUco marker detection (Section 4.5.3). While this ensures reliable marker localization, it creates a throughput bottleneck: a 100 m mission requires ≈ 21 minutes. Notably, these time inefficiencies arise primarily from the randomness of exploration movements (stopping, turning, backtracking) rather than the robot's nominal speed. A purely deterministic coverage path at the same linear velocity would be substantially faster.

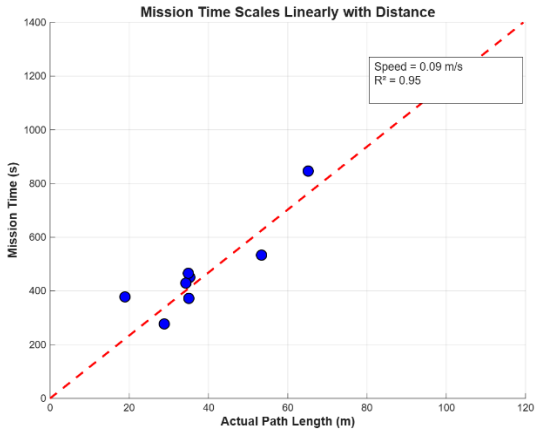



Figure 20: Mission time versus actual path length

## 5.2 Discussion

### 5.2.1 Interpretation of Results

The experimental data validates that the hybrid architecture achieves high coverage (≥96%) using only visual markers and wheel odometry, demonstrating the viability of mapless operation without LiDAR. However, a critical limitation emerged in the form of a bimodal efficiency distribution: in 30% of missions, repeated bumper activations in confined spaces created recovery loops, dropping path efficiency to $\varepsilon \approx 0.10$. For practical deployment in cluttered real-world environments, this limitation requires modifying the safety layer to incorporate either a collision-memory log, an intelligent escape routine, or adaptive velocity reduction near obstacles.

### 5.2.2 Comparison with the State of the Art

The developed architecture directly addresses the critical operational trade-offs found in traditional Coverage Path Planning (CPP). Unlike classical cellular decomposition, this system operates entirely online without prior workspace mapping. Compared to grid-based methods, it eliminates the high memory overhead of dense occupancy grids and the need for expensive

ranging sensors. Furthermore, while heuristic approaches often sacrifice coverage completeness for execution speed, this hybrid strategy achieves near-complete coverage while bypassing the intensive offline pre-computation required by sampling or optimization-based alternatives.

## 5.3 Conclusions

### 5.3.1 Summary of Contributions

This work provides three main contributions to online robotic coverage. First, it introduces a dynamic workspace reconstruction methodology utilizing embedded visual sensors and ArUco landmarks, eliminating the requirement for a priori environmental maps. Second, it implements a two-stage path planning algorithm that bridges the gap between high deterministic coverage guarantees and low computational overhead. Finally, it validates this framework through empirical physical deployment on a TurtleBot 4 platform, proving real-time algorithmic execution viability on resource-constrained embedded hardware.

### 5.3.2 Achievement of Research Objectives

The experimental results provide validation for the five research objectives established at the outset of this work. First, testing in unknown environments demonstrated that hybrid architecture can execute coverage planning online without prior maps. Second, operating without dense grids or SLAM allowed real-time execution on standard hardware, lowering computational demands. Third, the system maintained operational viability under a restricted sensor suite consisting only of a single RGB camera and wheel odometry. Fourth, the framework's performance was systematically quantified across ten trials using standard metrics, including coverage ratio and path efficiency. Finally, comparative analysis positions this bimodal approach as a practical alternative that balances coverage completeness with low computational and sensing requirements.

### 5.3.3 Recommendations for Future Work

Future research will focus on algorithmic optimization and refactoring the prototype into a native ROS 2 system (C++ or Python). This migration eliminates the MATLAB-ROS 2 bridge latency, maximizing frame rates for responsive, real-time obstacle handling. Key algorithmic enhancements include replacing the bumper loop with an adaptive state machine, fusing visual and wheel odometry, and supporting non-convex boundaries. Finally, expanding testing to low-light and open exterior environments will refine the control logic to ensure reliable dual indoor-outdoor operation.

# Bibliography


[1] ‘Smart Robots Market Size, Share, Growth & Forecast 2035’, Research Nester. Accessed: Feb. 18, 2026. [Online]. Available: https://www.researchnester.com/reports/smart-robots-market/4788

[2] H. González Jorge, L. M. González de Santos, N. Fariñas Álvarez, J. Martínez Sánchez, and F. Navarro Medina, ‘Operational Study of Drone Spraying Application for the Disinfection of Surfaces against the COVID-19 Pandemic’, *Drones*, vol. 5, no. 1, p. 18, Mar. 2021, doi: 10.3390/drones5010018.

[3] U. R. Mogili and B. B. V. L. Deepak, ‘Review on Application of Drone Systems in Precision Agriculture’, *Procedia Computer Science*, vol. 133, pp. 502–509, Jan. 2018, doi: 10.1016/j.procs.2018.07.063.

[4] V. Yogalakshmi, R. Jagadeeswaran, D. Muthumanickam, K. Thirukumaran, and M. Kavitha, ‘Application of agricultural drones in vegetable cultivation: A comprehensive review’, 2025, Accessed: Feb. 18, 2026. [Online]. Available: https://agris.fao.org/search/en/providers/122356/records/68010dd1dd564a9a8282fb6d

[5] J. Scharlemann, ‘Bundestag beschließt KRITIS-Dachgesetz’, Rosin Büdenbender. Accessed: Feb. 18, 2026. [Online]. Available: https://rosin-buedenbender.com/news/bundestag-beschliesst-kritis-dachgesetz/

[6] ‘Deutscher Bundestag Drucksache 21/2410 Gesetzentwurf der Bundesregierung Entwurf eines Gesetzes zur Umsetzung der Richtlinie (EU) 2022/2557 und zur Stärkung der Resilienz kritischer Anlagen’. Accessed: Feb. 18, 2026. [Online]. Available: https://dserver.bundestag.de/btd/21/025/2102510.pdf

[7] C. Wong, E. Yang, X.-T. Yan, and D. Gu, ‘An overview of robotics and autonomous systems for harsh environments’, in *2017 23rd International Conference on Automation and Computing (ICAC)*, Sep. 2017, pp. 1–6. doi: 10.23919/IConAC.2017.8082020.

[8] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, and W. Burgard, *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, 2005.

[9] C. S. Tan, R. Mohd-Mokhtar, and M. R. Arshad, ‘A Comprehensive Review of Coverage Path Planning in Robotics Using Classical and Heuristic Algorithms’, *IEEE Access*, vol. 9, pp. 119310–119342, 2021, doi: 10.1109/ACCESS.2021.3108177.

[10] H. Choset, ‘Coverage for robotics – A survey of recent results’, *Annals of Mathematics and Artificial Intelligence*, vol. 31, no. 1, pp. 113–126, Oct. 2001, doi: 10.1023/A:1016639210559.

[11] N. Agmon, N. Hazon, G. A. Kaminka, and The MAVERICK Group, ‘The giving tree: constructing trees for efficient offline and online multi-robot coverage’, *Ann Math Artif Intell*, vol. 52, no. 2, pp. 143–168, Apr. 2008, doi: 10.1007/s10472-009-9121-1.

[12] ‘2.5. Task Space and Workspace – Modern Robotics’. Accessed: Feb. 26, 2026. [Online]. Available: https://modernrobotics.northwestern.edu/nu-gm-book-resource/2-5-task-space-and-workspace/#department

[13] Lozano-Perez, ‘Spatial Planning: A Configuration Space Approach’, *IEEE Transactions on Computers*, vol. C–32, no. 2, pp. 108–120, Feb. 1983, doi: 10.1109/TC.1983.1676196.

[14] H. Choset, ‘Coverage of Known Spaces: The Boustrophedon Cellular Decomposition’, *Autonomous Robots*, vol. 9, no. 3, pp. 247–253, Dec. 2000, doi: 10.1023/A:1008958800904.

[15] ‘High resolution maps from wide angle sonar’. Accessed: Mar. 16, 2026. [Online]. Available: https://ieeexplore-1ieee-1org-100033ccr1172.han.technikum-wien.at/document/1087316

[16] E. Galceran and M. Carreras, ‘A survey on coverage path planning for robotics’, *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1258–1276, Dec. 2013, doi: 10.1016/j.robot.2013.09.004.

[17] A. Zelinsky, R. A. Jarvis, J. C. Byrne, and S. Yuta, 'Planning paths of complete coverage of an unstructured environment by a mobile robot', in *Proceedings of international conference on advanced robotics*, Citeseer, 1993, pp. 533–538. Accessed: Mar. 16, 2026. [Online]. Available: http://users.cecs.anu.edu.au/~rsl/rsl_papers/ICAR.ps
[18] 'Spiral-STC: an on-line coverage algorithm of grid environments by a mobile robot'. Accessed: Mar. 16, 2026. [Online]. Available: https://ieeexplore-1ieee-1org-100033ccr1179.han.technikum-wien.at/document/1013479
[19] 'A Comprehensive Review of Coverage Path Planning in Robotics Using Classical and Heuristic Algorithms'. Accessed: Mar. 18, 2026. [Online]. Available: https://ieeexplore-1ieee-1org-100033ccr1232.han.technikum-wien.at/document/9523743
[20] 'Path planning algorithm development for autonomous vacuum cleaner robots'. Accessed: Mar. 19, 2026. [Online]. Available: https://ieeexplore-1ieee-1org-100033co60034.han.technikum-wien.at/document/6850799
[21] L. M. Resler, 'Edward N Lorenz's 1963 paper, "Deterministic nonperiodic flow", in Journal of the Atmospheric Sciences, Vol 20, pages 130–141: Its history and relevance to physical geography', *Progress in Physical Geography: Earth and Environment*, vol. 40, no. 1, pp. 175–180, Feb. 2016, doi: 10.1177/0309133315623099.
[22] D. K. Sutantyo, S. Kernbach, P. Levi, and V. A. Nepomnyashchikh, 'Multi-robot searching algorithm using Lévy flight and artificial potential field', in *2010 IEEE Safety Security and Rescue Robotics*, Jul. 2010, pp. 1–6. doi: 10.1109/SSRR.2010.5981560.
[23] B. Yamauchi, 'A frontier-based approach for autonomous exploration', in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97. 'Towards New Computational Principles for Robotics and Automation'*, Jul. 1997, pp. 146–151. doi: 10.1109/CIRA.1997.613851.
[24] T.-K. Lee, S.-H. Baek, Y.-H. Choi, and S.-Y. Oh, 'Smooth coverage path planning and control of mobile robots based on high-resolution grid map representation', *Robotics and Autonomous Systems*, vol. 59, no. 10, pp. 801–812, Oct. 2011, doi: 10.1016/j.robot.2011.06.002.

# List of Figures

Figure 1: Gannt Diagram of the thesis schedule 8
Figure 2: Representation of a R^2 workspace (red) with obstacles (yellow). 9
Figure 3: Representation of a R^2 workspace with obstacles and robot (purple). 9
Figure 4: Representation of a R^2 workspace with obstacles, a robot, an expanded obstacle area, representations of footprints, and the trajectory q(t). 10
Figure 5: (a) Trapezoidal cellular decomposition of an environment (b) Trapezoidal adjacency graph 12
Figure 6: (a) Boustrophedon cellular decomposition of an environment (b) Boustrophedon adjacency graph 13
Figure 7: Representation of a grid-based cellular map 13
Figure 8: Representation of a wavefront map. A workspace (red) is populated by cells (blue) marked with numeralia. Cells with obstacles (yellow) are ignored. The robot (purple) sits at the bottom left. 14
Figure 9: (a) Pathing on first iteration of the algorithm, choosing the shortest route. (b) Pathing on second iteration. Previously visited cells are marked as obstacles, and furthest cell from origin is marked as goal. 14

Figure 10: A workspace (red) subdivided in mega cell (pink hatch) that are subdivided by small cells (red) 15
Figure 11: (a) Trajectory of a Spiral-STC algorithm that avoids megacells with obstacles inside. (b) Spanning tree node of megacells covered in the execution of this iteration 15
Figure 12: (a) to (d) showcase a step-by-step frontier-based exploration. 18
Figure 13: Mean Reprojection Error per Image, showing an overall mean error of 0.32 pixels., within the parameters of acceptance 25
Figure 14: Visualization of the scanned checkerboards taken on calibration in a 3D plot 26
Figure 15: Picture of the testing environment for the algorithm 27
Figure 16:Coverage ratio (η) per mission 28
Figure 17: Path efficiency ε frequency histogram 28
Figure 18: Redundancy (ρ) versus path efficiency (ε). No correlation exists 29
Figure 19: Theoretical vs Actual Energy Consumption of the 10 missions 29
Figure 20: Mission time versus actual path length 30

# List of Tables

Table 1: Performance metrics for coverage path planning evaluation 10
Table 2: Trade-offs of existing CPP approaches for unknown environments 11
Table 3: Comparisons of the different categories of CPP methods 20
Table 4: Performance metrics for the ten missions 27

# List of Abbreviations

| | |
|---|---|
| ACO | Ant Colony Optimization |
| APF | Artificial Potential Field |
| BFS | Breadth-First Search |
| BM | Brownian Motion |
| CPP | Coverage Path Planning |
| DFS | Depth-First Search |
| ESEIAAT | Escola Superior d'Enginyeries Industrial, Aeroespacial i Audiovisual de Terrassa |
| FOV | Field of View |
| GUI | Graphical User Interface |
| LF | Lévy Flight |
| PRM | Probabilistic Roadmaps |
| PSO | Particle Swarm Optimization |
| RGB | Red, Green, Blue |
| ROS/ROS2 | Robot Operating System |
| RRT/RRT* | Rapidly-Exploring Random Trees |

| | |
|---|---|
| RW | Random Walk |
| SLAM | Simultaneous Localization and Mapping |
| SNR | Signal-to-Noise Ratio |
| STC | Spanning Tree Coverage |
| TSP | Travelling Salesman Problem |
| UAS | University of Applied Sciences |
| UAV | Unmanned Aerial Vehicle |
| UPC | Universitat Politècnica de Catalunya |
| USD | United States Dollar |

# Documentation table of AI-based tools

| AI-based tools | Intended use | Prompt, source, page, paragraph... |
|---|---|---|
| **DeepSeek** | Grammar and spelling | "Please list issues with spelling and grammar in the following text: ..." Entire document |
| **DeepSeek** | Text cadences | “Review this text and tell me how to improve the redaction” |
| **DeepSeek** | Keyword searches | “I need to find a paper about X to cite my sources, what keywords should I use for Google Scholar?” |
| **Gemini** | Math Code Generation | “Based on this data, please generate me a graph for a figure supporting X statement” |
| **Gemini** | Pseudocode Generation | “Based on this code, generate a logic pseudocode flow to attach to my thesis” |
| **ChatGPT** | Academical audits | [Attached PDF Work in Progress] “Please audit this thesis so far, and help me understand weak points, needs of citation, or otherwise miscellaneous flaws.” |

# A: Repository Access and Version Control

The complete software implementation of the hybrid coverage path planning system, along with video demonstrations of the algorithm in action, is hosted on GitHub. This thesis has been presented on release version v.1.0.0.
Repository URL: https://github.com/VictorVictus/TurtleBot4-Online-CPP-Algorithm